Lecture
Developing Android Apps

Storing data locally using SQLite and Room API

Dr. Murad Mustafa Badarna

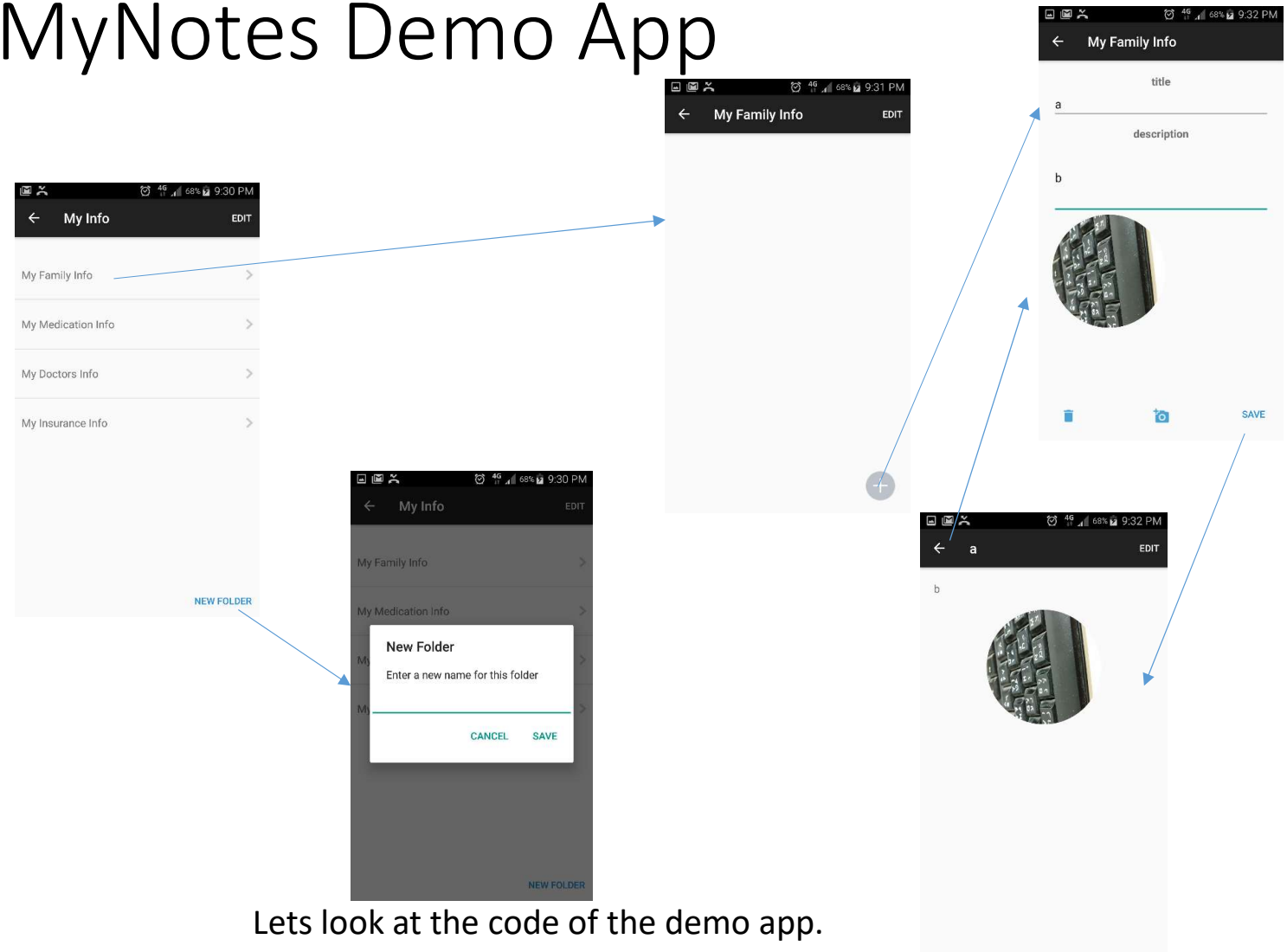
SQLite and Android

SQLiteהיא ספרייה קלת-משקל שמספקת מסד נתונים רלציוני מובנה המוטמע בתוך האפליקציה עצמה. כלומר, לא נדרש שרת מסד נתונים נפרד. היא תומכת בתחביר SQLמלא, טרנזקציות, ושאילתות מוכנות מראש.
היתרון הגדול של SQLiteהוא שהיא צורכת מעט מאוד זיכרון – בערך רבע מגה – מה שהופך אותה למתאימה במיוחד למכשירים ניידים.

- *SQLite* is an OpenSource database. SQLite supports standard relational database features like SQL syntax, transactions and prepared statements. The database requires limited memory at runtime (approx. 250 KByte) which makes it a good candidate from being embedded into other runtimes.
- SQLite is embedded into every Android device.
- Using an SQLite database in Android does not require a setup procedure or administration of the database.
- We must define the SQL statements for creating and updating the database.
- Afterwards the database is automatically managed for you by the Android platform.

כל מכשיר אנדרואיד כולל את SQLiteבאופן מובנה, ולכן אין צורך להתקין תוכנה נוספת.

MyNotes Demo App



מטרת האפליקציה היא להמחיש כיצד ניתן לשלב **SQLite** בתוך אפליקציה אמיתית, ולהשתמש בו לשמירה ושליפה של נתונים שהוזנו על ידי המשתמש. הממשק עצמו פשוט, אך מאחוריו פועלים תהליכים חשובים:

- יצירת טבלאות SQLite לאחסון התיקיות והפריטים
- שמירת נתונים באמצעות פעולות insert
- שליפה של נתונים להצגה במסכים
- שימוש ב־Adapter כדי להציג את הנתונים ברשימה

Lets look at the code of the demo app.

Import project called **MyNotes.zip** (download from our course website) into Android Studio

MyNotes SQLite structure

- **Notes Folder** – id (int), title (string)
- **Note Item** – id (int), title (string), description (string), image1 (bit map/byte[]), folderId (int)
- Create a Database Using SQL Helper
  - Once you have defined how your database looks, you should implement methods that create and maintain the database and tables.
  - A useful set of APIs is available in the `SQLiteOpenHelper` class.
  - When you use this class to obtain references to your database, the system performs the potentially long-running operations of creating and updating the database only when needed and *not during app startup*.
  - All you need to do is call `getWritableDatabase()` or `getReadableDatabase()`.

טבלה: Notes Folder
טבלה זו מייצגת את התיקיות שמכילות את ההערות. לכל תיקייה יש:
- id מספר מזהה ייחודי מסוג מספר שלם
- title כותרת התיקייה מסוג טקסט

טבלה: Note Item
טבלה זו מייצגת את ההערות עצמן. לכל הערה יש:
- id מזהה ייחודי מסוג מספר שלם
- title כותרת ההערה
- description תיאור ההערה
- image1 תמונה המאוחסנת כ- byte array או Bitmap
- folderId מזהה של התיקייה שההערה שייכת אליה, לקישור לטבלת Notes Folder

Create a Database Using a SQL Helper

כדי לנהל את מסד הנתונים ב־ Android בצורה נוחה ומסודרת, יש לרשת (extend) את המחלקה SQLiteOpenHelper ולממש שלוש מתודות עיקריות:

- **onCreate** מופעלת כאשר מסד הנתונים נוצר בפעם הראשונה
- **onUpgrade** מופעלת כאשר יש שינוי בגרסת מסד הנתונים(DATABASE_VERSION)
- **onOpen** מופעלת בכל פעם שהמסד נפתח לדוגמה, בעת קריאה או כתיבה

```
public class MyInfoDatabase extends SQLiteOpenHelper {

   private static final int DATABASE_VERSION = 1;
   private static final String DATABASE_NAME = "MyInfoDB";

   // folder table
   private static final String TABLE_FOLDER_NAME = "folders";
   private static final String FOLDER_COLUMN_ID = "id";
   private static final String FOLDER_COLUMN_TITLE = "title";
   private static final String[] TABLE_FOLDER_COLUMNS = {FOLDER_COLUMN_ID, FOLDER_COLUMN_TITLE};
```

Create a Database Using a SQL Helper

```
public class MyInfoDatabase extends SQLiteOpenHelper {
   ...
   // item table
   private static final String TABLE_ITEMS_NAME = "items";
   private static final String ITEM_COLUMN_ID = "id";
   private static final String ITEM_COLUMN_NAME = "title";
   private static final String ITEM_COLUMN_DESCRIPTION =
"description";
   private static final String ITEM_COLUMN__IMAGE1 = "image1";
   private static final String ITEM_COLUMN_FOLDERID = "folder_id";
   private static final String[] TABLE_ITEM_COLUMNS =
{ITEM_COLUMN_ID, ITEM_COLUMN_NAME,
         ITEM_COLUMN_DESCRIPTION, ITEM_COLUMN_IMAGE1,
ITEM_COLUMN_FOLDERID};


public MyInfoDatabase(Context context) {
   super(context, DATABASE_NAME, null, DATABASE_VERSION);

}
```

במקטע זה מוגדרת מחלקת MyInfoDatabase שיורשת מהמחלקה SQLiteOpenHelper, ומטרתה היא לנהל את יצירת הטבלאות ומסד הנתונים של האפליקציה.

כאן מוגדרים שמות הטבלה ועמודותיה בצורה קבועה. הגדרה כזו עוזרת:
- למנוע שגיאות כתיב חוזרות בקוד
- להקל על תחזוקה עתידית
- להבטיח עקביות בין שלבים שונים בקוד

DATABASE_NAME הוא שם הקובץ של מסד הנתונים שיישמר בזיכרון הקבוע של המכשיר DATABASE_VERSION מציין את גרסת מסד הנתונים (ונדרש לעדכונים עתידיים)

```
SQLiteOpenHelper (Context context, String name, SQLiteDatabase.CursorFactory factory, int version)
SQLiteDatabase.CursorFactory: to use for creating cursor objects, or null for the default
```

Cursor הוא אובייקט שמאפשר לעבור על תוצאות שאילתות של SELECT . כברירת מחדל, Android יוצרת עבורך Cursor סטנדרטי

Override `onCreate()` - SQLiteOpenHelper

Called when the database is created for the first time.
This is where the creation of tables and the initial population of the tables should happen.

המתודה מופעלת באופן אוטומטי כאשר מסד הנתונים נוצר בפעם הראשונה. זהו המקום שבו יוצרים את הטבלאות הדרושות לאפליקציה.

```
@Override
public void onCreate(SQLiteDatabase db) {
   try {
      // SQL statement to create item table
      String CREATE_ITEM_TABLE = "create table if not exists " + TABLE_ITEMS_NAME +"
( "
            + ITEM_COLUMN_ID +" INTEGER PRIMARY KEY AUTOINCREMENT, "
            + ITEM_COLUMN_NAME +" TEXT, "
            + ITEM_COLUMN_DESCRIPTION + " TEXT, "
            + ITEM_COLUMN__IMAGE1 + " BLOB, "
            + ITEM_COLUMN_FOLDERID + " INTEGER)";
      db.execSQL(CREATE_ITEM_TABLE);

          // SQL statement to create item table
         String CREATE_FOLDER_TABLE = "create table if not exists "+
TABLE_FOLDER_NAME+" ( "
               + FOLDER_COLUMN_ID +" INTEGER PRIMARY KEY AUTOINCREMENT, "
               + FOLDER_COLUMN_TITLE + " TEXT)";

         db.execSQL(CREATE_FOLDER_TABLE);
      }

   } catch (Throwable t) {
      t.printStackTrace();
   }
}
```

Override `onUpgrade()` - `SQLiteOpenHelper`

- Called when the database needs to be upgraded. The implementation should use this method to drop tables, add tables, or do anything else it needs to upgrade to the new schema version.
- If you add new columns you can use ALTER TABLE to insert them into a live table.
- If you rename or remove columns you can use ALTER TABLE to rename the old table, then create the new table and then populate the new table with the contents of the old table.

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    try {
        // drop item table if already exists
        //db.execSQL("DROP TABLE IF EXISTS items");
        //db.execSQL("DROP TABLE IF EXISTS folders");
    } catch (Throwable t) {
        t.printStackTrace();
    }

}
```

מופעלת אוטומטית כאשר גרסת מסד הנתונים (DATABASE_VERSION) משתנה – לדוגמה, אם נעלה את הגרסה מ- 1ל-2.
זהו המקום לבצע שינויים מבניים כמו:

- הוספת טבלאות
- שינוי שמות עמודות
- הוספת עמודות חדשות
- מחיקת טבלאות קיימות

Cursor - methods

```
Cursor cursor = db.rawQuery("SELECT * FROM students");
cursor.moveToFirst();
do {
    int id = cursor.getInt(cursor.getColumnIndex("id"));
    String email = cursor.getString(
                        cursor.getColumnIndex("email"));
    ...
} while (cursor.moveToNext());
cursor.close();
```

- Cursor lets you iterate through row results one at a time
  - getBlob(*index*), getColumnCount(), getColumnIndex(name), getColumnName(*index*), getCount(), getDouble(*index*), getFloat(*index*), getInt(*index*), getLong(*index*), getString(*index*), moveToPrevious(), ...

SQLite – select sql

```
public List<InfoFolder> getAllFolders() {
   List<InfoFolder> result = new ArrayList<InfoFolder>();
   Cursor cursor = null;
   try {
      cursor = db.query(TABLE_FOLDER_NAME, TABLE_FOLDER_COLUMNS, null, null,
            null, null, null);

      cursor.moveToFirst();
      while (!cursor.isAfterLast()) {
         InfoFolder folder = cursorToFolder(cursor);
         result.add(folder);
         cursor.moveToNext();
      }

   } catch (Throwable t) {
      t.printStackTrace();
   }
   finally {
      // make sure to close the cursor
      if(cursor!=null){
         cursor.close();
      }
   }
   return result;
}
```

```
public class InfoFolder {
   private int id;
   private String title;
```

```
Cursor query (String table, String[] columns, String
selection, String[] selectionArgs,
String groupBy, String having, String orderBy
, String limit)
```

```
private InfoFolder cursorToFolder(Cursor cursor) {
   InfoFolder result = new InfoFolder();
   try {
      result.setId(cursor.getInt(0));
      result.setTitle(cursor.getString(1));
   } catch (Throwable t) {
      t.printStackTrace();
   }
   return result;
}
```

SQLite – select sql

Query the given table, returning a `Cursor` over the result set.

Parameters	
`table`	`String`: The table name to compile the query against.
`columns`	`String`: A list of which columns to return. Passing null will return all columns, which is discouraged to prevent reading data from storage that isn't going to be used.
`selection`	`String`: A filter declaring which rows to return, formatted as an SQL WHERE clause (excluding the WHERE itself). Passing null will return all rows for the given table.
`selectionArgs`	`String`: You may include ?s in selection, which will be replaced by the values from selectionArgs, in order that they appear in the selection. The values will be bound as Strings.
`groupBy`	`String`: A filter declaring how to group rows, formatted as an SQL GROUP BY clause (excluding the GROUP BY itself). Passing null will cause the rows to not be grouped.
`having`	`String`: A filter declare which row groups to include in the cursor, if row grouping is being used, formatted as an SQL HAVING clause (excluding the HAVING itself). Passing null will cause all row groups to be included, and is required when row grouping is not being used.
`orderBy`	`String`: How to order the rows, formatted as an SQL ORDER BY clause (excluding the ORDER BY itself). Passing null will use the default sort order, which may be unordered.
`limit`	`String`: Limits the number of rows returned by the query, formatted as LIMIT clause. Passing null denotes no LIMIT clause.

Returns	
`Cursor`	A `Cursor` object, which is positioned before the first entry. Note that `Cursor`s are not synchronized, see the documentation for more details.

SQLite – select sql by parameter

```
public List<InfoItem> getAllItemsOfFolder(InfoFolder folder) {
   List<InfoItem> result = new ArrayList<InfoItem>();
   Cursor cursor = null;
   try {
      int floderId = folder.getId();
      cursor = db.query(TABLE_ITEMS_NAME, TABLE_ITEM_COLUMNS, ITEM_COLUMN_FOLDERID +" = ?",
            new String[] { String.valueOf(floderId) }, null, null,
            null, null);
      if(cursor!=null && cursor.getCount()>0){
         cursor.moveToFirst();
         while (!cursor.isAfterLast()) {
            InfoItem item = cursorToItem(cursor);
            result.add(item);
            cursor.moveToNext();
         }
      }
   } catch (Throwable t) {
      t.printStackTrace();
   }
   finally {
      if(cursor!=null) {
         // make sure to close the cursor
         cursor.close();
      }
   }
   return result;
}
```

SQLite – select sql by parameter

```
private InfoItem cursorToItem(Cursor cursor) {
   InfoItem result = new InfoItem();
   try {
      result.setId(cursor.getInt(0));
      result.setTitle(cursor.getString(1));
      result.setDescription(cursor.getString(2));
      //images
      byte[] img1Byte = cursor.getBlob(3);
      if (img1Byte != null && img1Byte.length > 0) {
         Bitmap image1 = BitmapFactory.decodeByteArray(img1Byte, 0, img1Byte.length);
         if (image1 != null) {
            result.setImage1(image1);
         }
      }
       result.setFolderId(cursor.getInt(4)));
   } catch (Throwable t) {
      t.printStackTrace();
   }
   return result;
}
```

```
public class InfoItem {
   private int id;
   private String title;
   private String description;
   private Bitmap image1 = null;
   private int folderId = -1;
```

SQLite – insert

```
public void createFolder(InfoFolder folder) {
    try {
        // make values to be inserted
        ContentValues values = new ContentValues();
        values.put(FOLDER_COLUMN_TITLE, folder.getTitle());
        // insert folder
        db.insert(TABLE_FOLDER_NAME, null, values);
    } catch (Throwable t) {
        t.printStackTrace();
    }
}
```

```
long insert (String table,
             String nullColumnHack,
             ContentValues values)
```

Convenience method for inserting a row into the database.

Parameters	
table	String: the table to insert the row into
nullColumnHack	String: optional; may be null. SQL doesn't allow inserting a completely empty row without naming at least one column name. If your provided values is empty, no column names are known and an empty row can't be inserted. If not set to null, the nullColumnHack parameter provides the name of nullable column name to explicitly insert a NULL into in the case where your values is empty.
values	ContentValues: this map contains the initial column values for the row. The keys should be the column names and the values the column values

Returns	
long	the row ID of the newly inserted row, or -1 if an error occurred

משמש במקרה שבו ריקContentValues – צריך לציין עמודה כלשהי כדי לאפשר הכנסת שורה ריקה

SQLite – update

```
public int updateFolder(InfoFolder folder) {
   int i = 0;
   try {
// make values to be inserted
      ContentValues values = new ContentValues();
      values.put(FOLDER_COLUMN_TITLE, folder.getTitle());
      // update
      i = db.update(TABLE_FOLDER_NAME, values, FOLDER_COLUMN_ID
+ " = ?",
            new String[] { String.valueOf(folder.getId()) });
   } catch (Throwable t) {
      t.printStackTrace();
   }
   return i;
}
```

update

```
int update (String table,
                ContentValues values,
                String whereClause,
                String[] whereArgs)
```

Convenience method for updating rows in the database.

Parameters	
table	String: the table to update in
values	ContentValues: a map from column names to new column values. null is a valid value that will be translated to NULL.
whereClause	String: the optional WHERE clause to apply when updating. Passing null will update all rows.
whereArgs	String: You may include ?s in the where clause, which will be replaced by the values from whereArgs. The values will be bound as Strings.

Returns	
int	the number of rows affected

SQLite – delete

```
public void deleteItem(InfoItem item) {

    try {

        // delete item
        db.delete(TABLE_ITEMS_NAME, ITEM_COLUMN_ID + " = ?",
                new String[] { String.valueOf(item.getId()) });
    } catch (Throwable t) {
        t.printStackTrace();
    }

}
```

delete

```
int delete (String table,
                String whereClause,
                String[] whereArgs)
```

Convenience method for deleting rows in the database.

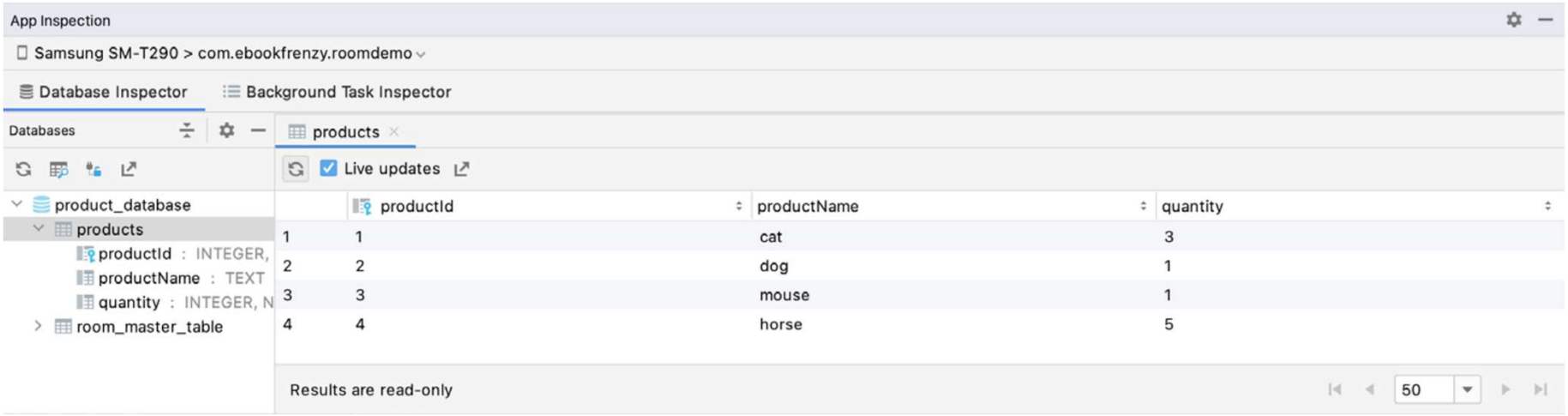
Parameters	
table	String: the table to delete from
whereClause	String: the optional WHERE clause to apply when deleting. Passing null will delete all rows.
whereArgs	String: You may include ?s in the where clause, which will be replaced by the values from whereArgs. The values will be bound as Strings.

Returns	
int	the number of rows affected if a whereClause is passed in, 0 otherwise. To remove all rows and get a count pass "1" as the whereClause.

More info about all operations can be found here:
https://developer.android.com/reference/android/database/sqlite/SQLiteDatabase.html

Database Inspector

- Android Studio includes a Database Inspector tool window which allows the databases associated with running apps to be viewed, searched and modified



Design Pattern: Singleton (1)

דפוס Singleton נועד להבטיח שקיימת רק מופע אחד של מחלקה מסוימת לאורך חיי האפליקציה, ושניתן לגשת אליו בקלות מכל מקום בקוד.

- The singleton design pattern solves problems like:
  - How can it be ensured that a class has only one instance?
    - Control class instantiation
  - How can the sole instance of a class be accessed easily?
    - Restrict the number of instances of a class

שימושים נפוצים באנדרואיד
•ניהול חיבור למסד נתונים
•ניהול הגדרות מערכת
•שירותים גלובליים כמו Analyticsאו Firebase
•מחלקות ניהול קונפיגורציה או תקשורת עם API

•איך נוודא שמחלקה נוצרה רק פעם אחת
•שומרים את הקונסטרקטור כפרטי(private)
•מונעים יצירת מופעים נוספים מחוץ למחלקה
•איך נוכל לגשת למופע היחיד בקלות
•יוצרים מתודה סטטית שמחזירה את המופע הקיים או יוצרת אותו אם עדיין לא קיים

Design Pattern: Singleton (2)

- The singleton design pattern describes how to solve such problems:
  - Hide the constructor of the class.
    - The hidden constructor (declared private) ensures that the class can never be instantiated from outside the class.
  - Define a public static operation (getInstance()) that returns the sole instance of the class.
    - make the class itself responsible for controlling its instantiation (that it is instantiated only once).
  - The public static operation can be accessed easily by using the class name and operation name (Singleton.getInstance()).

דפוס Singleton – איך מיישמים בפועל

שלב 1 – הסתרת הבנאי Constructor

שלב 2 – יצירת מתודה סטטית ציבורית שמחזירה את המופע היחיד

שלב 3 – שימוש בדפוס

גישה למופע מתבצעת בצורה סטטית דרך שם המחלקה

DataBase controller – operation, open and close

```
public class MyInfoManager {
   private static MyInfoManager instance = null;
   private Context context = null;
   private MyInfoDatabase db = null;
   private InfoFolder selectedFolder = null;
   private InfoItem selectedItem = null;


   public static MyInfoManager getInstance() {
         if (instance == null) {
            instance = new MyInfoManager();
         }
         return instance;
      }

public void openDataBase(Context context) {
   this.context = context;
   if (context != null) {
      db = new MyInfoDatabase(context);
      db.open();
   }
}
public void closeDataBase() {
   if(db!=null){
      db.close();
   }
}
```

```
public List<InfoFolder> getAllFolders() {
   List<InfoFolder> result = new ArrayList<InfoFolder>();
   if (db != null) {
      result = db.getAllFolders();
   }
   return result;
}
```

```
public class MyInfoDatabase extends
SQLiteOpenHelper {

public void open() {
   try {
      db = getWritableDatabase();
   } catch (Throwable t) {
      t.printStackTrace();
   }
}

public void close() {
   try {
      db.close();
   } catch (Throwable t) {
      t.printStackTrace();
   }
}
```

```
public class MainActivity
@Override
protected void onCreate(..) {
   super.onCreate(savedInstanceState);
    MyInfoManager.getInstance()
         .openDataBase(this);


}

@Override
protected void onDestroy() {
    MyInfoManager.getInstance()
         .closeDataBase();
    super.onDestroy();
}
```

Save data in a local database using Room

- The actual SQLite database responsible for storing and providing access to the data
- The app code, including the repository, should never make direct access to this underlying database
- All database operations are performed using a combination of the room database, DAOs and entities
- .

Roomהיא ספריית ORM (Object Relational Mapping)שמספקת שכבת הפשטה מעל SQLite. היא מאפשרת לכתוב קוד קריא, מודרני ובטוח יותר – תוך שימוש במחלקות ישירות במקום בשאילתות גולמיות

עקרונות חשובים של Room

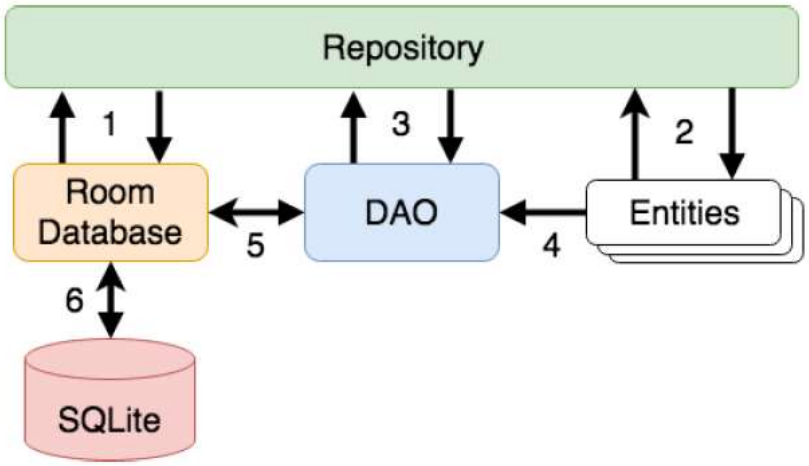
1.הקוד באפליקציה לא ניגש ישירות למסד הנתונים

כל גישה לנתונים נעשית דרך Repositoryשמתקשר עם מחלקות DAO.

DAO, Entity, RoomDatabase.2

שלושת הרכיבים הבסיסיים לעבודה עם Room:

1. **Entity** – מחלקה שמייצגת טבלה במסד הנתונים
2. **DAO** – מחלקה עם מתודות אנוטציה (כגון insert, update, query)
3. **RoomDatabase** – המחלקה הראשית שמחברת בין ה־ DAOלמסד הנתונים



More details: https://developer.android.com/training/data-storage/room

Entities

Simple entity

```
@Entity(tableName = "products")
public class ProductEntity {

@PrimaryKey
@ColumnInfo(name = "id")
private long id;

@ColumnInfo(name = "name")
private String name;

@ColumnInfo(name = "description")
private String description;

@ColumnInfo(name = "qty")
private long quantity;
```

Foreign keys

```
@Entity(
   tableName = "inventory",
   foreignKeys = {
      @ForeignKey(
         entity = ProductEntity.class,
         parentColumns = {"id"},
         childColumns = {"product_id"}
      )
   }
)
```

```
public class InventoryEntity {

   @PrimaryKey
   private long id;

   @ColumnInfo(name = "provider_id")
   private long providerId;

   @ColumnInfo(name = "product_id")
   private long productId;

   @ColumnInfo(name = "qty")
   private long quantity;
```

Nested objects

```
@Entity(tableName = "products")
public class ProductEntity {

    @PrimaryKey
    @ColumnInfo(name = "id")
    private long id;

    @ColumnInfo(name = "name")
    private String name;

    @ColumnInfo(name = "description")
    private String description;

    @Embedded
    private ProductInfo info;
```

```
public class ProductInfo {

    @ColumnInfo(name = "color")
    private String color;

    @ColumnInfo(name = "size")
    private String size;
```

DAOs

Simple DAO

```
@Dao
public interface ProductsDao {

    @Insert(onConflict = REPLACE)
    long insert(ProductEntity productEntity);

    @Query("SELECT * FROM products")
    List<ProductEntity> getAll();

    @Update
    int update(ProductEntity productEntity);

    @Delete
    int delete(ProductEntity productEntity);
}
```

ProductsDao – פעולות בסיסיות מול טבלת המוצרים

ממשק DAO ב־Room המאפשר לבצע פעולות CRUD על טבלת `products`:

- `insert` – הוספת מוצר חדש או החלפת מוצר קיים
- `getAll` – שליפת כל המוצרים מהטבלה
- `update` – עדכון מוצר קיים
- `delete` – מחיקת מוצר מהטבלה

הממשק מפריד בין קוד האפליקציה לבין הגישה הישירה למסד הנתונים.

Insert

```
@Dao
public interface ProductsDao {

    @Insert(onConflict = REPLACE)
    long insert(ProductEntity product);

    @Insert(onConflict = REPLACE)
    List<Long> insertAll(ProductEntity... products);

    @Insert(onConflict = ROLLBACK)
    List<Long> insertTwo(ProductEntity product, ProductEntity otherProduct);

    @Insert(onConflict = ABORT)
    List<Long> weirdInsert(ProductEntity product, List<ProductEntity> products);
}
```

Update

```
import androidx.room.Dao;
import androidx.room.Update;

@Dao
public interface ProductsDao {

   @Update
   int update(ProductEntity product);

   @Update
   int update(ProductEntity... products);
}
```

Delete

```
import androidx.room.Dao;
import androidx.room.Delete;

@Dao
public interface ProductsDao {

    @Delete
    int delete(ProductEntity product);

    @Delete
    int delete(ProductEntity... products);
}
```

Query

ProductsDao – שליפת מוצרים בעזרת שאילתות

ממשק DAO ב־Room הכולל שאילתות שונות לשליפת נתונים מטבלת `products`:

- `getAllAsList` – מחזירה את כל המוצרים כרשימה
- `getAllAsArray` – מחזירה את כל המוצרים כמערך
- `getProductsWithName` – מחזירה מוצרים לפי שם מסוים
- `getProductsByName` – מחזירה מוצרים לפי רשימת שמות

השאילתות מאפשרות לשלוף נתונים מהמסד לפי תנאים שונים בצורה פשוטה ומובנית.

```
@Dao
public interface ProductsDao {

    @Query("SELECT * FROM products")
    List<ProductEntity> getAllAsList();

    @Query("SELECT * FROM products")
    ProductEntity[] getAllAsArray();

    @Query("SELECT * FROM products WHERE name = :name")
    List<ProductEntity> getProductsWithName(String name);

    @Query("SELECT * FROM products WHERE name IN (:names)")
    List<ProductEntity> getProductsByName(List<String> names);
}
```

Query

```
@Dao
public interface ProductsDao {

    @Query("SELECT * FROM products WHERE id = :id")
    ProductEntity getProducts(long id);

    @Query("SELECT COUNT(*) FROM products")
    int getProductsCount();

    @Query("SELECT COUNT(*) AS count, name FROM products GROUP BY name")
    List<ProductCountEntity> getProductsCountByName();
}
```

Database

Database

```
@Database(
   entities = {
      ProductEntity.class,
      InventoryEntity.class
   },
   version = 1
)

public abstract class MyDatabase extends
RoomDatabase {


   public abstract ProductsDao productsDao();

   public abstract InventoryDao inventoryDao();
}
```

MyDatabase – הגדרת מסד הנתונים ב־Room

המחלקה `MyDatabase` מגדירה את מסד הנתונים של האפליקציה באמצעות Room.

- `@Database` – מציינת שזו מחלקת מסד נתונים
- `entities` – מגדיר אילו טבלאות יהיו במסד הנתונים: `ProductEntity` ו־`InventoryEntity`
- `version = 1` – מגדיר את גרסת מסד הנתונים
- `extends RoomDatabase` – מאפשר למחלקה לעבוד כמסד נתונים של Room
- `productsDao()` ו־`inventoryDao()` – מספקות גישה לממשקי ה־DAO של המוצרים והמלאי

המחלקה מרכזת את ההגדרות של מסד הנתונים ואת הגישה לטבלאות השונות.

Using database

Use a single instance of database!

```
MyDatabase database = Room
        .databaseBuilder(
                getApplicationContext(),
                MyDatabase.class,
                "database"
        )
        .build();

List<ProductEntity> products = database.productsDao().getAllAsList();
```

LiveData

```
import androidx.lifecycle.LiveData;
@Query("SELECT * FROM products")
LiveData<List<ProductEntity>> getAllAsList();
```

שליפת מוצרים עם LiveData

המתודה `getAllAsList` שולפת את כל המוצרים מטבלת `products` ומחזירה אותם בתוך `LiveData`.

- `@Query` – מגדירה את שאילתת השליפה מהטבלה
- `LiveData<List<ProductEntity>>` – מחזיר רשימת מוצרים שניתן לצפות בה
- כאשר הנתונים בטבלה משתנים, Room מעדכן אוטומטית את ה־`LiveData` מתאים במיוחד להצגת נתונים במסך שמתעדכן אוטומטית כאשר מסד הנתונים משתנה.

השוואה בין Room ו SQLiteOpenHelper

Room	SQLiteOpenHelper	נושא
גבוהה ((High-level	נמוכה ((Low-level	רמת עבודה
דרך אנוטציות	ידנית	כתיבת SQL
יש	אין	בדיקות בזמן קומפילציה
קלה	מורכבת	תחזוקה
נמוך	גבוה	Boilerplate code

מתי להשתמש בכל אחד?

SQLiteOpenHelper

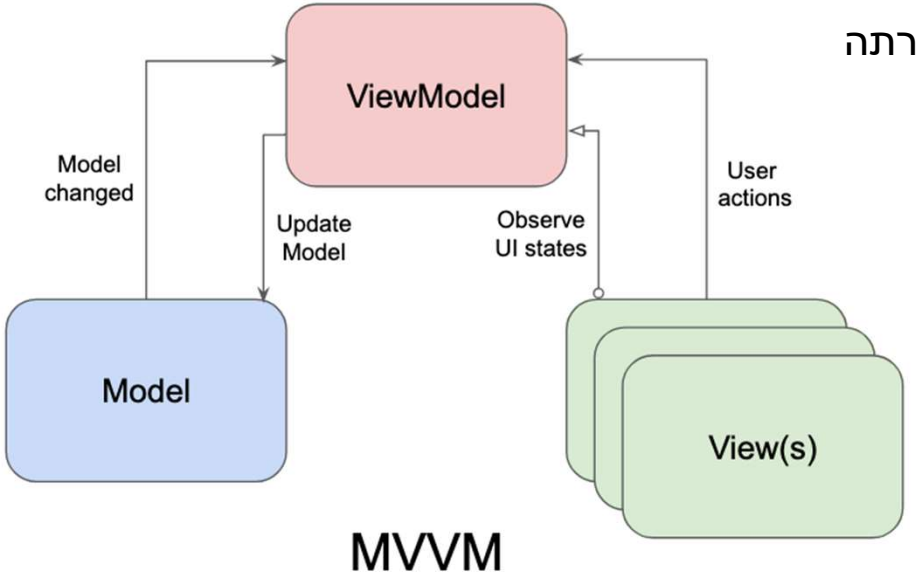
- כשצריך שליטה מלאה
- ביצועים מאוד ספציפיים

Room

- אפליקציות מודרניות
- שילוב עם MVVM

Room + MVVM Architecture

עיקרון תכנוני
Separation of Concerns
הפרדת אחריות בין שכבות



מהי ארכיטקטורת MVVM?
MVVM (Model-View-ViewModel)היא תבנית תכנון שמטרתה להפריד בין:

- לוגיקת הממשק View
  - מציג נתונים למשתמש
  - מאזין לשינויים בנתונים LiveData
  - לא מבצע לוגיקה עסקי
  - לא ניגש ישירות למסד נתונים
- לוגיקת הנתונים ViewModel
- מתווך בין Viewל־Model
- מחזיק ומנהל את הנתונים עבור ה־UI
- שכבת הנתונים Model
- אחראית על ניהול הנתונים באפליקציהכוללת:מסד נתונים Room / SQLite

Room + MVVM Architecture

זרימת הנתונים

1. המשתמש מבצע פעולה ב־UI
2. ה־View פונה ל־ViewModel
3. ה־ViewModel מבקש נתונים מה־Repository
   ***תפקיד ה־Repository** שכבת ביניים בין ViewModel ל־Data Sources
4. ה־Repository פונה ל־DAO (Room)
5. הנתונים חוזרים כ־LiveData / Flow
6. ה־UI מתעדכן אוטומטית

מה זה חשוב באנדרואיד?

- אפליקציות מודרניות דורשות:
  - עבודה אסינכרונית
  - עדכון UI בזמן אמת
  - ניהול נכון של lifecycle

Background Operations ו־ Threading

```
Executor executor = Executors.newSingleThreadExecutor();

executor.execute(() -> {
   database.dao().insert(item);
});
```

UIנשאר חלק ומהיר
חוויית משתמש טובה יותר
מניעת קריסות ואזהרות מערכת

- Main Thread אחראי על ה־UI
- ציור המסך
- תגובה ללחיצות משתמש
- פעולות DBהן איטיות יחסית

קריאה/כתיבה למסד נתונים עלולות לקחת זמן

אם נריץ אותן על ה־Main Thread

- האפליקציה תיתקע UI Freeze
- המשתמש לא יוכל ללחוץ/לגלול
- עלול להופיע: **ANR (Application Not Responding)**